

```
In [44]: #yfinance is a popular Python library used for downloading historical market data f  
#It simplifies the process of accessing financial data for various securities, incl  
  
#pip install yfinance
```

```
In [6]: import seaborn as sns  
import yfinance as yf  
import numpy as np  
import pandas as pd  
import matplotlib.pyplot as plt  
from sklearn.model_selection import train_test_split  
from sklearn.ensemble import RandomForestRegressor  
#from sklearn import metrics  
#from sklearn.metrics import accuracy_score, classification_report, confusion_matri
```

```
In [9]: btc = yf.Ticker('BTC-USD')  
prices1 = btc.history(period='5y')  
prices1.drop(columns=['Open', 'High', 'Low', 'Dividends', 'Stock Splits'], axis = 1)  
  
eth = yf.Ticker('ETH-USD')  
prices2 = eth.history(period='5y')  
prices2.drop(columns=['Open', 'High', 'Low', 'Dividends', 'Stock Splits'], axis = 1)  
  
usdt = yf.Ticker('USDT-USD')  
prices3 = usdt.history(period='5y')  
prices3.drop(columns=['Open', 'High', 'Low', 'Dividends', 'Stock Splits'], axis = 1)  
  
bnb = yf.Ticker('BNB-USD')  
prices4 = bnb.history(period='5y')  
prices4.drop(columns=['Open', 'High', 'Low', 'Dividends', 'Stock Splits'], axis = 1)
```

```
In [5]: #The parameters lsuffix and rsuffix in the join method are used to add suffixes to  
# This is necessary to avoid column name conflicts when the two DataFrames have col  
  
p1 = prices1.join(prices2, lsuffix = ' (BTC)', rsuffix = ' (ETH)')  
p2 = prices3.join(prices4, lsuffix = ' (USDT)', rsuffix = ' (BNB)')  
data = p1.join(p2, lsuffix = '_', rsuffix = '_')
```

```
In [11]: data.head()
```

Out[11]:

	Close (BTC)	Volume (BTC)	Close (ETH)	Volume (ETH)	Close (USDT)	Volume (USD)
Date						
2021-01-05 00:00:00+00:00	33992.429688	67547324782	1100.006104	41535932781	1.002202	101918715
2021-01-06 00:00:00+00:00	36824.363281	75289433811	1207.112183	44699914188	1.001528	116105139
2021-01-07 00:00:00+00:00	39371.042969	84762141031	1225.678101	40468027280	1.000400	129467601
2021-01-08 00:00:00+00:00	40797.609375	88107519480	1224.197144	44334826666	1.000045	131555961
2021-01-09 00:00:00+00:00	40254.546875	61984162837	1281.077271	33233105361	1.002947	103292241



In [12]: data.tail()

Out[12]:

	Close (BTC)	Volume (BTC)	Close (ETH)	Volume (ETH)	Close (USDT)	Volume (USD)
Date						
2025-12-31 00:00:00+00:00	87508.828125	33830210616	2967.037598	16451891101	0.998449	702594611
2026-01-01 00:00:00+00:00	88731.984375	18849043990	3000.394287	10268796662	0.998745	505486662
2026-01-02 00:00:00+00:00	89944.695312	46398906171	3124.422607	25242778003	0.999672	961285663
2026-01-03 00:00:00+00:00	90603.187500	20774828592	3125.917480	11460707919	0.999635	556601048
2026-01-05 00:00:00+00:00	92551.148438	33869225984	3163.892090	16618956800	0.999493	769940029



In [14]: data.shape

Out[14]: (1826, 8)

In [15]: data.info()

```
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 1826 entries, 2021-01-05 00:00:00+00:00 to 2026-01-05 00:00:00+00:00
Data columns (total 8 columns):
 #   Column                Non-Null Count  Dtype
---  -
 0   Close (BTC)           1826 non-null   float64
 1   Volume (BTC)          1826 non-null   int64
 2   Close (ETH)           1826 non-null   float64
 3   Volume (ETH)          1826 non-null   int64
 4   Close (USDT)          1826 non-null   float64
 5   Volume (USDT)         1826 non-null   int64
 6   Close (BNB)           1826 non-null   float64
 7   Volume (BNB)          1826 non-null   int64
dtypes: float64(4), int64(4)
memory usage: 128.4 KB
```

In [16]: `data.isna().sum()`

```
Out[16]: Close (BTC)      0
Volume (BTC)      0
Close (ETH)       0
Volume (ETH)      0
Close (USDT)      0
Volume (USDT)     0
Close (BNB)       0
Volume (BNB)      0
dtype: int64
```

In [17]: `data.describe()`

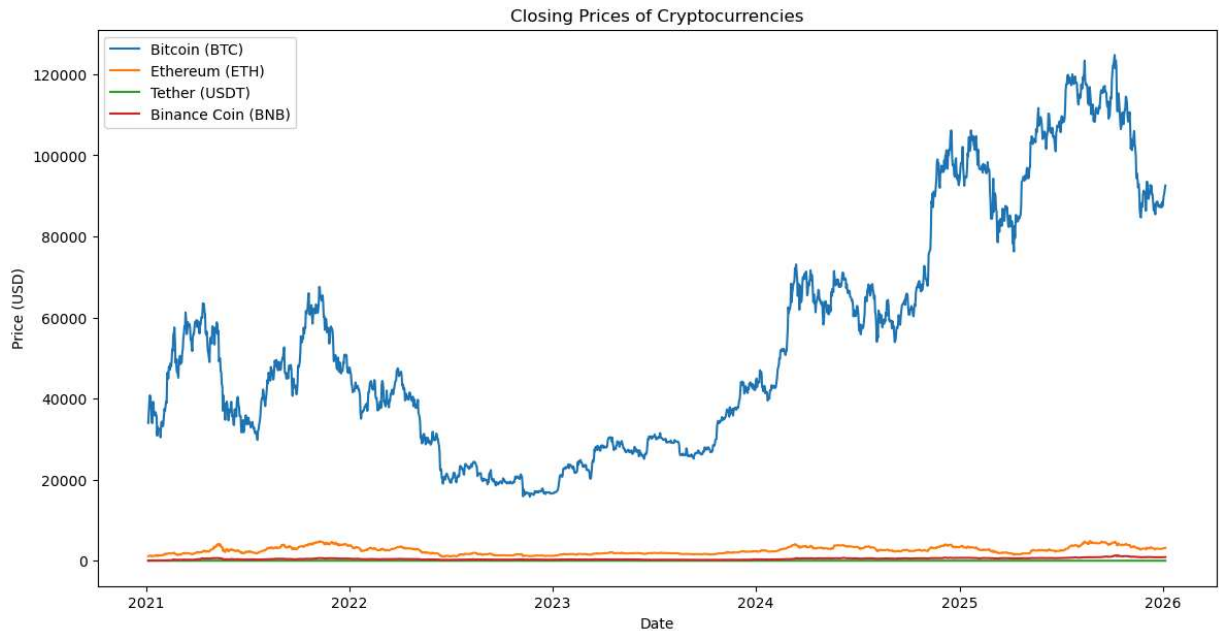
```
Out[17]:
```

	Close (BTC)	Volume (BTC)	Close (ETH)	Volume (ETH)	Close (USDT)	Volume (USDT)
count	1826.000000	1.826000e+03	1826.000000	1.826000e+03	1826.000000	1.826000e+03
mean	54555.368174	3.709425e+10	2539.663464	1.949922e+10	1.000145	7.086675e+10
std	29492.368568	2.300411e+10	908.613040	1.290797e+10	0.000710	4.501654e+10
min	15787.284180	5.331173e+09	993.636780	2.081626e+09	0.995872	9.989859e+09
25%	29410.323730	2.130374e+10	1793.623047	1.025222e+10	0.999893	4.009190e+10
50%	46617.654297	3.176162e+10	2457.574585	1.643440e+10	1.000142	6.035691e+10
75%	69333.382812	4.703862e+10	3235.419373	2.501231e+10	1.000384	8.805684e+10
max	124752.531250	3.509679e+11	4831.348633	9.773662e+10	1.011530	3.443980e+11

Exploratory Data Analysis

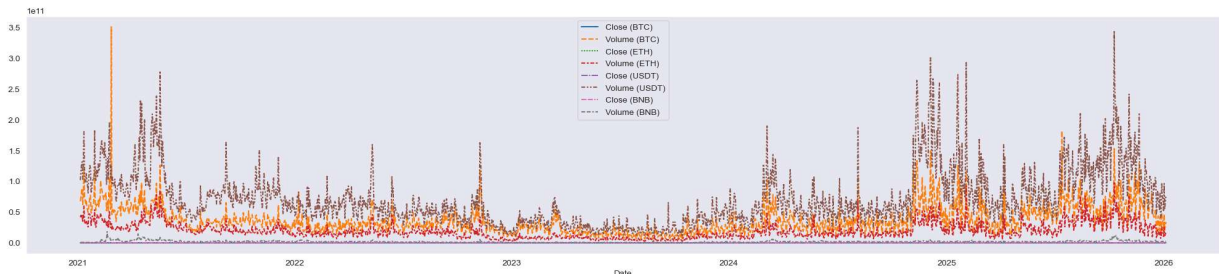
```
In [18]: #Visualize the Closing Prices
# create a line plot to visualize the closing prices of all four cryptocurrencies o
```

```
plt.figure(figsize=(14, 7))
plt.plot(data.index, data['Close (BTC)'], label='Bitcoin (BTC)')
plt.plot(data.index, data['Close (ETH)'], label='Ethereum (ETH)')
plt.plot(data.index, data['Close (USDT)'], label='Tether (USDT)')
plt.plot(data.index, data['Close (BNB)'], label='Binance Coin (BNB)')
plt.title('Closing Prices of Cryptocurrencies')
plt.xlabel('Date')
plt.ylabel('Price (USD)')
plt.legend()
plt.show()
```



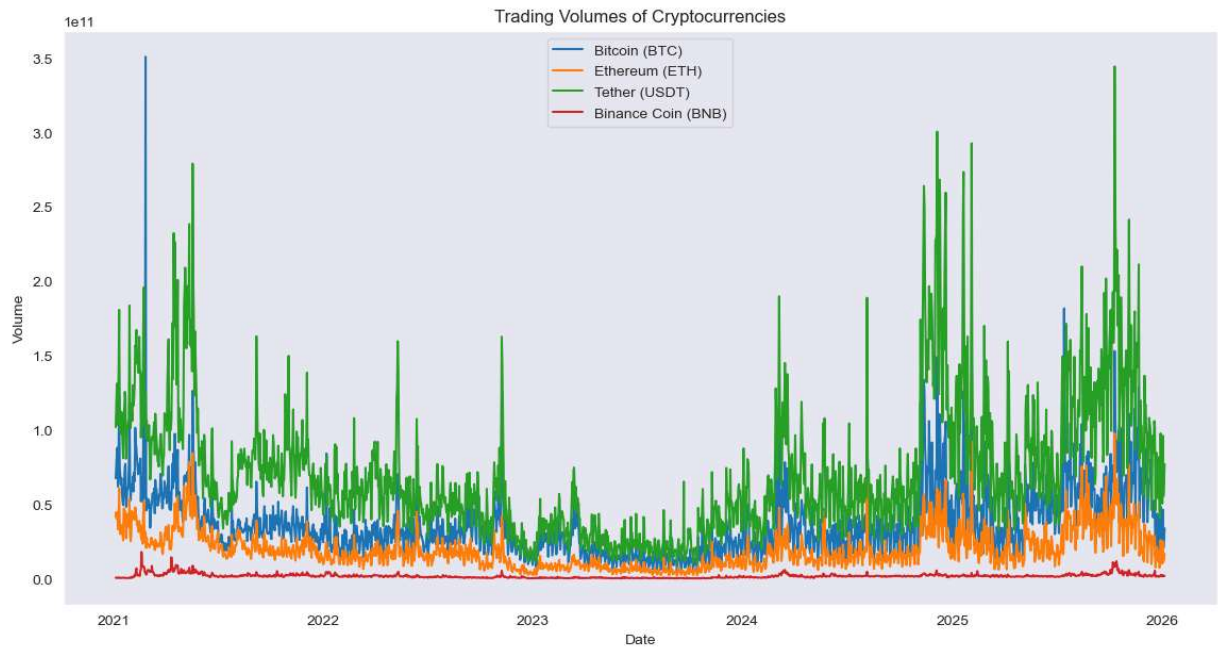
```
In [19]: plt.figure(figsize = (25, 5))
sns.set_style('dark')
sns.lineplot(data=data)
```

```
Out[19]: <Axes: xlabel='Date'>
```



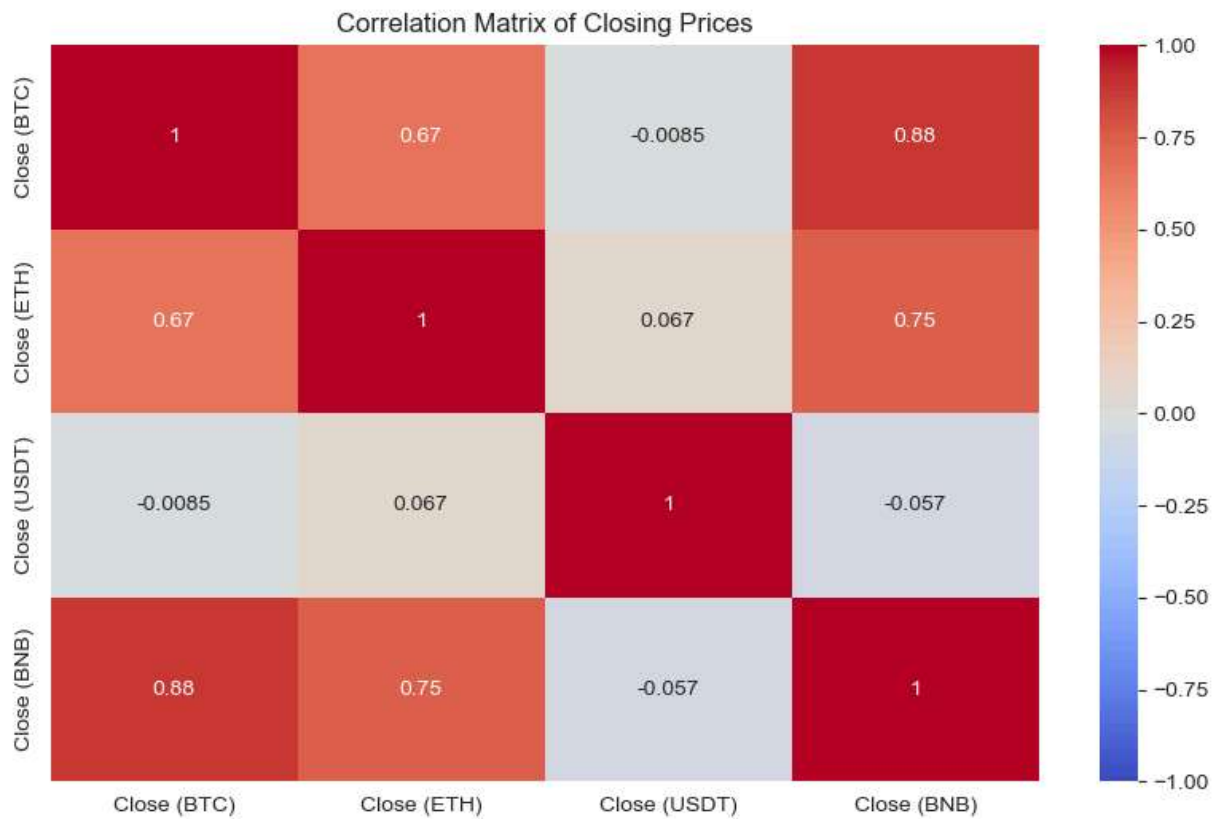
```
In [20]: # Visualize the Trading Volumes
#Let's visualize the trading volumes of all four cryptocurrencies:
plt.figure(figsize=(14, 7))
plt.plot(data.index, data['Volume (BTC)'], label='Bitcoin (BTC)')
plt.plot(data.index, data['Volume (ETH)'], label='Ethereum (ETH)')
plt.plot(data.index, data['Volume (USDT)'], label='Tether (USDT)')
plt.plot(data.index, data['Volume (BNB)'], label='Binance Coin (BNB)')
plt.title('Trading Volumes of Cryptocurrencies')
plt.xlabel('Date')
plt.ylabel('Volume')
```

```
plt.legend()
plt.show()
```



```
In [21]: #Correlation Analysis
#We'll analyze the correlation between the closing prices of the cryptocurrencies:
# Calculate the correlation matrix
corr_matrix = data[['Close (BTC)', 'Close (ETH)', 'Close (USDT)', 'Close (BNB)']].c

# Plot the heatmap
plt.figure(figsize=(10, 6))
sns.heatmap(corr_matrix, annot=True, cmap='coolwarm', vmin=-1, vmax=1)
plt.title('Correlation Matrix of Closing Prices')
plt.show()
```



```
In [7]: # Distribution of Closing Prices
#Let's plot the distribution of closing prices for each cryptocurrency:
plt.figure(figsize=(14, 7))

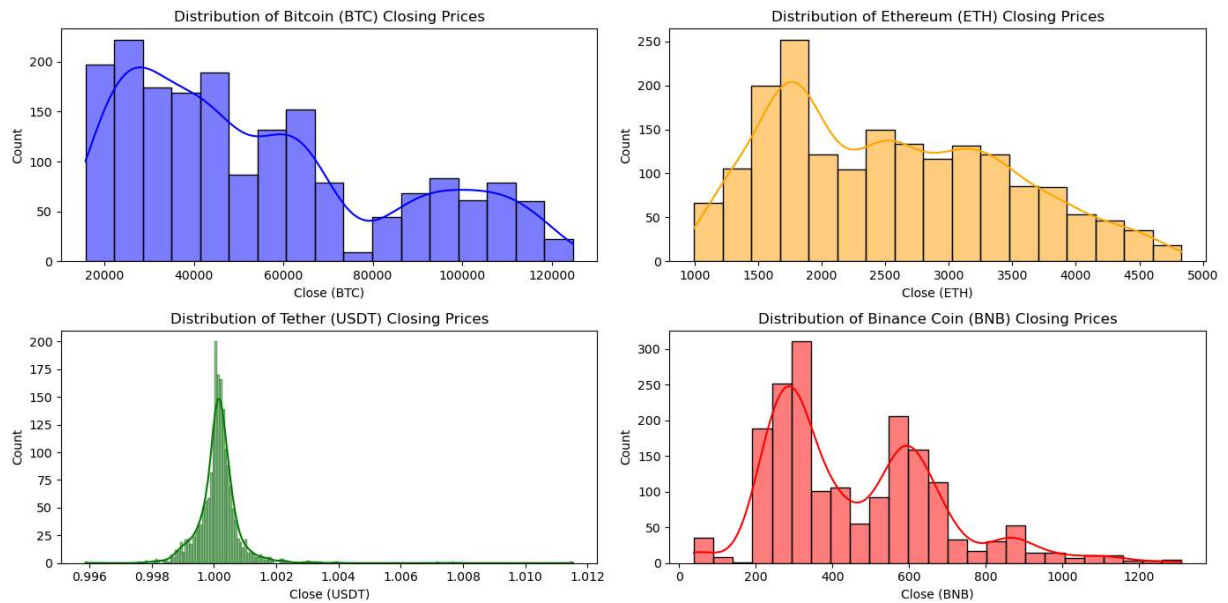
plt.subplot(2, 2, 1)
sns.histplot(data['Close (BTC)'], kde=True, color='blue')
plt.title('Distribution of Bitcoin (BTC) Closing Prices')

plt.subplot(2, 2, 2)
sns.histplot(data['Close (ETH)'], kde=True, color='orange')
plt.title('Distribution of Ethereum (ETH) Closing Prices')

plt.subplot(2, 2, 3)
sns.histplot(data['Close (USDT)'], kde=True, color='green')
plt.title('Distribution of Tether (USDT) Closing Prices')

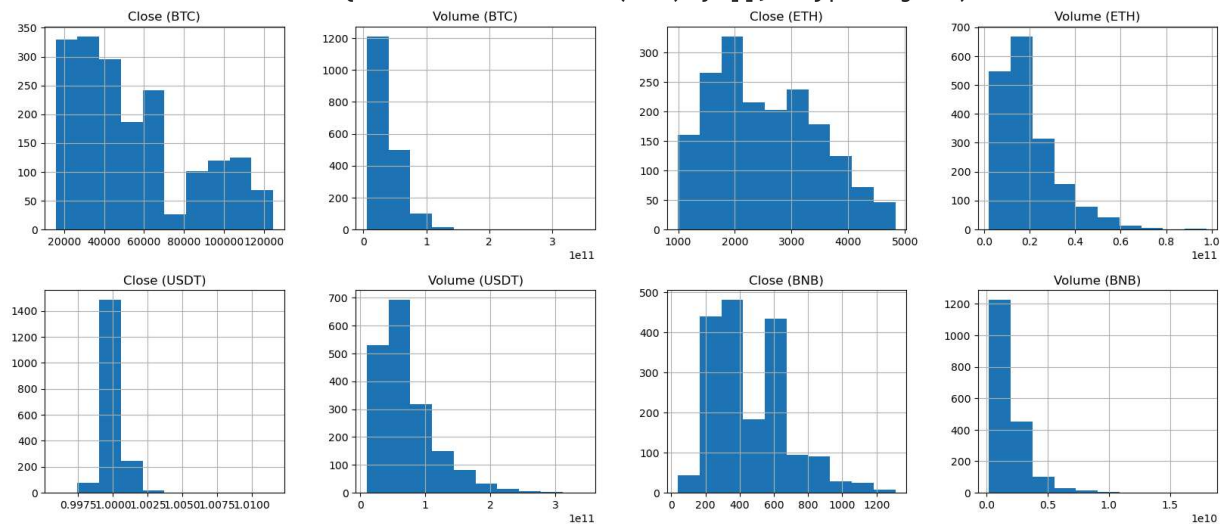
plt.subplot(2, 2, 4)
sns.histplot(data['Close (BNB)'], kde=True, color='red')
plt.title('Distribution of Binance Coin (BNB) Closing Prices')

plt.tight_layout()
plt.show()
```



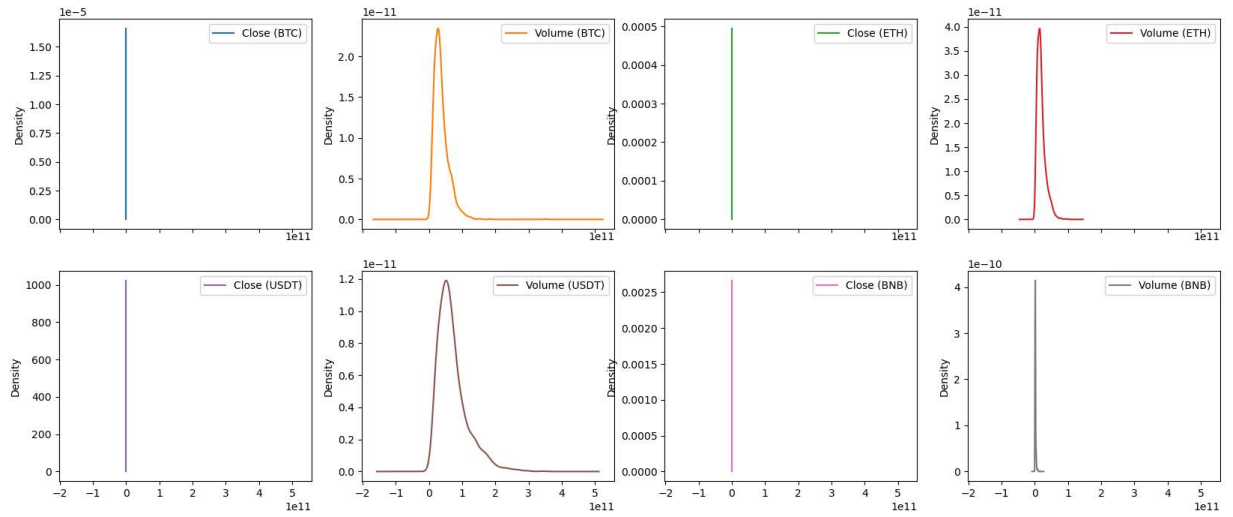
```
In [8]: data.hist(figsize=(20, 8), layout=(2, 4))
```

```
Out[8]: array([[<Axes: title={'center': 'Close (BTC)'>,
<Axes: title={'center': 'Volume (BTC)'>,
<Axes: title={'center': 'Close (ETH)'>,
<Axes: title={'center': 'Volume (ETH)'>],
[<Axes: title={'center': 'Close (USDT)'>,
<Axes: title={'center': 'Volume (USDT)'>,
<Axes: title={'center': 'Close (BNB)'>,
<Axes: title={'center': 'Volume (BNB)'>]], dtype=object)
```



```
In [12]: data.plot(kind = "kde", subplots = True, layout = (2, 4), figsize = (20, 8))
```

```
Out[12]: array([[<Axes: ylabel='Density'>, <Axes: ylabel='Density'>,
<Axes: ylabel='Density'>, <Axes: ylabel='Density'>],
[<Axes: ylabel='Density'>, <Axes: ylabel='Density'>,
<Axes: ylabel='Density'>, <Axes: ylabel='Density'>]], dtype=object)
```

```
In [26]: sns.pairplot(data.sample(n=100));
```



Data Pre-Processing

```
In [13]: X = data.drop(columns = ['Close (BTC)'], axis = 1)
Y = data.loc[:, 'Close (BTC)']
```

```
In [14]: X.head()
```

Out[14]:

	Volume (BTC)	Close (ETH)	Volume (ETH)	Close (USDT)	Volume (USDT)	Close (BNB)
Date						
2021-01-05 00:00:00+00:00	67547324782	1100.006104	41535932781	1.002202	101918715244	41.734600
2021-01-06 00:00:00+00:00	75289433811	1207.112183	44699914188	1.001528	116105139289	42.165955
2021-01-07 00:00:00+00:00	84762141031	1225.678101	40468027280	1.000400	129467601516	43.449490
2021-01-08 00:00:00+00:00	88107519480	1224.197144	44334826666	1.000045	131555961745	42.395317
2021-01-09 00:00:00+00:00	61984162837	1281.077271	33233105361	1.002947	103292241007	43.932854



```
In [15]: X.tail()
```

Out[15]:

	Volume (BTC)	Close (ETH)	Volume (ETH)	Close (USDT)	Volume (USDT)	Close (BNB)
Date						
2026-01-01 00:00:00+00:00	18849043990	3000.394287	10268796662	0.998745	50548666268	863.054626
2026-01-02 00:00:00+00:00	46398906171	3124.422607	25242778003	0.999672	96128566387	880.844177
2026-01-03 00:00:00+00:00	20774828592	3125.917480	11460707919	0.999635	55660104876	878.639465
2026-01-04 00:00:00+00:00	26770491368	3140.710449	13890479323	0.999506	66989138853	894.383667
2026-01-05 00:00:00+00:00	33925971968	3154.499268	17201358848	0.999383	77426376704	896.885010



```
In [31]: Y.head()
```

```
Out[31]: Date
2021-01-05 00:00:00+00:00    33992.429688
2021-01-06 00:00:00+00:00    36824.363281
2021-01-07 00:00:00+00:00    39371.042969
2021-01-08 00:00:00+00:00    40797.609375
2021-01-09 00:00:00+00:00    40254.546875
Name: Close (BTC), dtype: float64
```

```
In [16]: # Split the data into training and testing sets
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_sta
```

```
In [35]: # Split the data into training and testing sets
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_sta
```

```
In [19]: # Print the shapes of the resulting datasets
print(f'X_train shape: {X_train.shape}')
print(f'X_test shape: {X_test.shape}')
print(f'y_train shape: {Y_train.shape}')
print(f'y_test shape: {Y_test.shape}')
```

```
X_train shape: (1461, 7)
X_test shape: (366, 7)
y_train shape: (1461,)
y_test shape: (366,)
```

```
In [37]: from sklearn.feature_selection import SelectKBest
fs = SelectKBest(k=4)
X_train = fs.fit_transform(X_train, Y_train)
X_test = fs.transform(X_test)
```

```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\feature_selection\_univariate_sel
ection.py:109: RuntimeWarning: invalid value encountered in divide
msw = sswn / float(dfwn)
```

```
In [38]: mask = fs.get_support()
selected_features = X.columns[mask]
print("Selected Features:", selected_features)
```

```
Selected Features: Index(['Close (USDT)', 'Volume (USDT)', 'Close (BNB)', 'Volume (B
NB)'], dtype='object')
```

```
In [39]: X_train
```

```
Out[39]: array([[1.00007701e+00, 1.47110596e+11, 6.89921143e+02, 2.16720240e+09],
 [9.99653995e-01, 1.81473614e+11, 6.13587280e+02, 2.63725732e+09],
 [9.99648988e-01, 9.18534158e+10, 6.61424561e+02, 1.65908213e+09],
 ...,
 [1.00041604e+00, 1.36735818e+11, 7.08126099e+02, 2.29864945e+09],
 [9.99777019e-01, 7.03906069e+10, 2.64886047e+02, 1.63232454e+09],
 [9.98982012e-01, 3.38348894e+10, 2.64235138e+02, 7.20361905e+08]])
```

```
In [40]: #MinMaxScaler is a preprocessing method in scikit-learn that transforms features by
# It's often used when your data needs to be normalized within a specific range to
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
```

```
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [41]: *# implementation of 10 different regression algorithms using scikit-learn. Each alg*

#Import Libraries and Generate Sample Data

```
from sklearn.datasets import make_regression
from sklearn.linear_model import LinearRegression, Ridge, Lasso, ElasticNet
from sklearn.svm import SVR
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor, GradientBoostingRegressor
from sklearn.neighbors import KNeighborsRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, r2_score
```

In [42]: *#Define Models and Perform Training and Evaluation*

```
models = {
    'Linear Regression': LinearRegression(),
    'Ridge Regression': Ridge(alpha=1.0),
    'Lasso Regression': Lasso(alpha=1.0),
    'ElasticNet Regression': ElasticNet(alpha=1.0, l1_ratio=0.5),
    'Support Vector Regression (SVR)': SVR(kernel='rbf'),
    'Decision Tree Regression': DecisionTreeRegressor(),
    'Random Forest Regression': RandomForestRegressor(n_estimators=100),
    'Gradient Boosting Regression': GradientBoostingRegressor(n_estimators=100, lea
    'K-Nearest Neighbors Regression': KNeighborsRegressor(n_neighbors=5),
    'Neural Network Regression (MLP)': MLPRegressor(hidden_layer_sizes=(100, 50), a
}

# Train and evaluate each model
results = {'Model': [], 'MSE': [], 'R-squared': []}

for name, model in models.items():
    # Train the model
    model.fit(X_train, Y_train)

    # Predict on test set
    Y_pred = model.predict(X_test)

    # Evaluate model
    mse = mean_squared_error(Y_test, Y_pred)
    r2 = r2_score(Y_test, Y_pred)

    # Store results
    results['Model'].append(name)
    results['MSE'].append(mse)
    results['R-squared'].append(r2)

    # Print results
    print(f"----- {name} -----")
    print(f"Mean Squared Error (MSE): {mse}")
    print(f"R-squared: {r2}")
    print()
```

```
# Convert results to DataFrame for visualization
results_df = pd.DataFrame(results)
print(results_df)

# Plotting the results
plt.figure(figsize=(12, 6))
plt.barh(results_df['Model'], results_df['R-squared'], color='skyblue')
plt.xlabel('R-squared')
plt.title('R-squared of Different Regression Models')
plt.xlim(-1, 1)
plt.gca().invert_yaxis()
plt.show()
```

----- Linear Regression -----

Mean Squared Error (MSE): 147933237.82801753

R-squared: 0.8280492111393685

----- Ridge Regression -----

Mean Squared Error (MSE): 149349055.7958133

R-squared: 0.826403529479047

----- Lasso Regression -----

Mean Squared Error (MSE): 147993911.58717552

R-squared: 0.8279786867534799

----- ElasticNet Regression -----

Mean Squared Error (MSE): 763746876.4105973

R-squared: 0.11225577282826837

----- Support Vector Regression (SVR) -----

Mean Squared Error (MSE): 909206254.8224329

R-squared: -0.05681951567565946

----- Decision Tree Regression -----

Mean Squared Error (MSE): 85135730.5883079

R-squared: 0.9010421440791775

----- Random Forest Regression -----

Mean Squared Error (MSE): 48857593.40905678

R-squared: 0.9432101814854742

----- Gradient Boosting Regression -----

Mean Squared Error (MSE): 55752889.524422005

R-squared: 0.9351954065513711

----- K-Nearest Neighbors Regression -----

Mean Squared Error (MSE): 52037454.32202047

R-squared: 0.9395140574738652

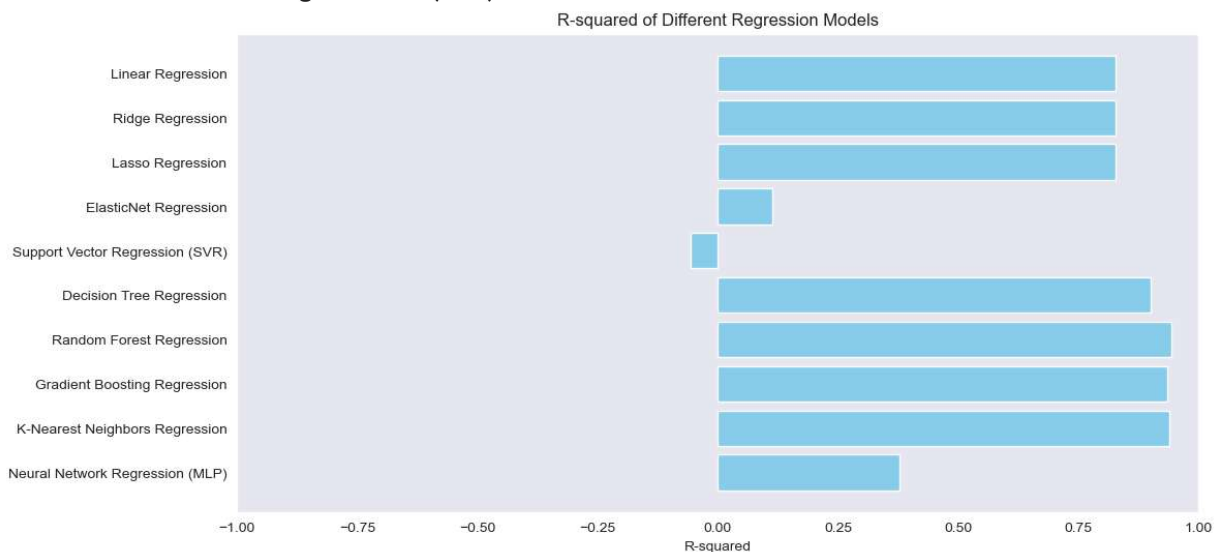
```
C:\ProgramData\anaconda3\Lib\site-packages\sklearn\neural_network\_multilayer_perceptron.py:690: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (200) reached and the optimization hasn't converged yet.
  warnings.warn(
```

----- Neural Network Regression (MLP) -----

Mean Squared Error (MSE): 536203571.3561882

R-squared: 0.3767416407678851

	Model	MSE	R-squared
0	Linear Regression	1.479332e+08	0.828049
1	Ridge Regression	1.493491e+08	0.826404
2	Lasso Regression	1.479939e+08	0.827979
3	ElasticNet Regression	7.637469e+08	0.112256
4	Support Vector Regression (SVR)	9.092063e+08	-0.056820
5	Decision Tree Regression	8.513573e+07	0.901042
6	Random Forest Regression	4.885759e+07	0.943210
7	Gradient Boosting Regression	5.575289e+07	0.935195
8	K-Nearest Neighbors Regression	5.203745e+07	0.939514
9	Neural Network Regression (MLP)	5.362036e+08	0.376742



Random Forest Regression is a powerful and versatile algorithm suitable for various regression tasks, offering robust performance and the ability to handle complex data relationships

Saving The Model

```
In [43]: import pickle
import numpy as np
from sklearn.datasets import make_regression
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_squared_error, r2_score

# Generate sample data
X, Y = make_regression(n_samples=1000, n_features=10, noise=0.1, random_state=0)

# Scale the features (optional but recommended for some algorithms)
```

```
scaler = MinMaxScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Initialize Random Forest Regressor
model_rf = RandomForestRegressor(n_estimators=100, random_state=0)

# Train the model
model_rf.fit(X_train, Y_train)

# Save the model to a file
filename = 'random_forest_model.pkl'
pickle.dump(model_rf, open(filename, 'wb'))

# Save scaler to a file
with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)

# Load the model from the file
loaded_model = pickle.load(open(filename, 'rb'))

# Predict using the Loaded model
Y_pred = loaded_model.predict(X_test)

# Evaluate the Loaded model
mse = mean_squared_error(Y_test, Y_pred)
r2 = r2_score(Y_test, Y_pred)

print(f"Loaded Random Forest Regression - Mean Squared Error (MSE): {mse}")
print(f"Loaded Random Forest Regression - R-squared: {r2}")
```

Loaded Random Forest Regression - Mean Squared Error (MSE): 48566815.08070425

Loaded Random Forest Regression - R-squared: 0.9435481688348887